



The Superior University, Lahore

LAB-TASK (9) (Spring-2026)

Course Title:	Programming for AI				Course Code:	-	Credit Hours	1
Instructor:	Sir rasikh				Program:	BSDS		
Semester:	4th	Batch:	Morning	Section:	4A	Date:	-	
Time Allowed:	-				Maximum Marks:		N/A	
Student’s Name:	Shumaila Maryam				Roll No.	SU92-BSDSM-S25-062		

Work Assigned:

Task 1; Submit any task from the given subtasks within the Section: Natural Language Processing Tasks (<https://www.geeksforgeeks.org/natural-language-processing-nlp-tutorial>)



DEPARTMENT OF
**SOFTWARE
ENGINEERING**

1. Overview

This task implements text summarization in Python. It starts with a transformer-based approach using the pre-trained t5-small model, then explores extractive summarization techniques provided by the Sumy library. The notebook is suitable for understanding basic NLP summarization workflows, model loading, tokenization, and sentence extraction algorithms.

2. Objectives

- Install and configure required NLP libraries.
- Load a pre-trained Hugging Face T5 tokenizer and model.
- Prepare text input using the summarization prefix expected by T5.
- Generate and decode an abstractive summary.
- Use Sumy for tokenization, stemming, and extractive summarization.
- Compare Luhn, Edmundson, and LSA summarization techniques.

3. Software Requirements

Package / Tool	Purpose	Installation
Python	Programming language used for the notebook.	Install Python 3.x
Jupyter Notebook	Interactive environment for running cells.	pip install notebook
transformers	Hugging Face model and tokenizer classes.	pip install transformers
torch	Backend framework for running T5 model inference.	pip install torch
sumy	Extractive text summarization algorithms.	pip install sumy
nltk	Sentence tokenization resources required by Sumy.	pip install nltk

4. Notebook Workflow

Step 1: Install dependencies: The notebook installs the transformer stack and Sumy library. These provide model-based and algorithm-based summarization support.

Step 2: Import model classes: T5Tokenizer converts text into tokens. T5ForConditionalGeneration loads the generation model that produces summaries.

Step 3: Load pre-trained model: The model name t5-small is used. It is lightweight compared with larger T5 variants and works well for demonstration purposes.

Step 4: Prepare input text: The input text is prefixed with summarize: so the T5 model understands that the task is summarization.

Step 5: Tokenize input: The tokenizer converts the text into PyTorch tensors and truncates long input to a maximum length of 512 tokens.

Step 6: Generate summary: The model.generate function creates a summarized sequence using beam search and length constraints.

Step 7: Decode summary: The generated token IDs are converted back into readable text and printed.

Step 8: Apply Sumy methods: The notebook then demonstrates tokenization, stemming, Luhn, Edmundson, and LSA extractive summarization.

5. Hugging Face T5 Summarization

This section implements abstractive summarization. Abstractive summarization can generate new wording instead of only selecting sentences from the original text.

5.1 Installation

Before using the T5 model, the required Python libraries must be installed. The main library used for this task is Transformers, developed by Hugging Face. It provides pre-trained models and tokenizers for Natural Language Processing tasks such as text summarization, translation, question answering, and text generation.

The PyTorch library is also required because the T5 model uses deep learning operations to process input text and generate summaries.

```
%pip install transformers torch
```

Python

5.2 Import Required Classes

After installation, the required classes are imported from the Transformers library. The T5Tokenizer is responsible for converting human-readable text into tokens that the model can understand. Since machine learning models cannot directly process raw text, the tokenizer breaks the text into smaller units and converts them into numerical IDs.

The T5ForConditionalGeneration class loads the T5 model architecture for text generation tasks. It is used for tasks where the model generates an output sequence based on an input sequence, such as summarization.

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
```

Python

5.3 Load Model and Tokenizer

In this step, the pre-trained T5 model and tokenizer are loaded. Here, t5-small is used because it is a lightweight version of the T5 model. It requires fewer computational resources and is suitable for basic summarization tasks, experiments, and learning purposes.

The tokenizer is loaded using T5Tokenizer.from_pretrained(), which downloads or loads the tokenizer associated with the selected model. The model is loaded using T5ForConditionalGeneration.from_pretrained(), which provides the pre-trained weights needed to generate summaries.

Using a pre-trained model saves time because the model has already learned language patterns from a large amount of text data.

```
model_name = "t5-small"

tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)
```

Python

You are using the default legacy behaviour of the <class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>.

5.4 Prepare the Input Text

The input text is the paragraph that needs to be summarized.

T5 follows a text-to-text approach, meaning both input and output are treated as text. To tell the model what task it needs to perform, a task prefix is added before the input text.

In this case, the prefix “summarize:” is added before the paragraph. This informs the T5 model that the required task is text summarization.

The final input given to the model is a combination of the task instruction and the original paragraph.

```
text = """
Artificial Intelligence is transforming industries across the globe.
From healthcare to finance, AI systems are automating processes,
analyzing data and improving decision making.
Organizations are investing heavily in AI research,
though ethical and privacy challenges remain.
"""

input_text = "summarize: " + text
```

Python

5.5 Tokenize and Generate Summary

Before a machine learning model can process text, it must first convert the text into a numerical format. This process is called **tokenization**. The tokenizer breaks the input text into smaller units (tokens) and assigns each token a unique numerical ID. These token IDs are then used as input for the model.

The `tokenizer.encode()` function performs this conversion. It also allows control over input size using parameters like `max_length`, which limits the number of tokens, and `truncation=True`, which ensures that extra text is removed if it exceeds the limit. The parameter `return_tensors="pt"` converts the tokens into PyTorch tensors so they can be processed by the model.

After tokenization, the model generates a summary using the `generate()` function. This function creates a shorter version of the original text by understanding its meaning. Parameters like `max_length` and `min_length` control the size of the summary, while `num_beams` improves quality using beam search, and `early_stopping=True` stops generation when a complete summary is formed.

Finally, the generated token IDs are converted back into human-readable text using `tokenizer.decode()`. The parameter `skip_special_tokens=True` removes unnecessary symbols, producing a clean and readable summary.

```
inputs = tokenizer.encode(
    input_text,
    return_tensors="pt",
    max_length=512,
    truncation=True
)
```

Python

6. Generate the Summary

```
summary_ids = model.generate(
    inputs,
    max_length=60,
    min_length=20,
    length_penalty=2.0,
    num_beams=4,
    early_stopping=True
)
```

Python

5.6 Key Generation Parameters

The quality and length of the generated summary depend on the generation parameters used in the `generate()` function.

max_length defines the maximum number of tokens allowed in the generated summary. In this example, the summary cannot exceed 60 tokens.

min_length defines the minimum number of tokens required in the generated summary. Here, the model is encouraged to generate at least 20 tokens.

length_penalty controls the length of the generated text. A higher value encourages the model to avoid very short summaries and produce more balanced output.

num_beams is used for beam search. Beam search helps the model explore multiple possible summaries and choose the best one. In this example, `num_beams=4` means the model considers four possible output paths while generating the summary.

early_stopping stops the generation process when the model has completed a suitable summary. This helps avoid unnecessary or repeated text generation.

Together, these parameters help control the summarization process and improve the quality of the final summary.

Parameter	Explanation
<code>max_length=60</code>	Limits the generated summary to 60 tokens.
<code>min_length=20</code>	Ensures the summary is not too short.
<code>length_penalty=2.0</code>	Discourages overly long generated sequences.
<code>num_beams=4</code>	Uses beam search to explore multiple candidate summaries.
<code>early_stopping=True</code>	Stops generation once the beam search has completed suitable candidates.

6. Sumy Extractive Summarization

Sumy provides extractive summarizers. Extractive summarization selects important sentences from the original document instead of generating entirely new sentences.

6.1 Installation and Tokenization

```
%pip install sumy
```

Python

```
from sumy.nlp.tokenizers import Tokenizer
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

# Create tokenizer for English
tokenizer = Tokenizer("en")

# Sample text
text = """Machine learning is transforming industries worldwide.
Companies are investing heavily in AI research and development.
The future of technology depends on these advancements."""

# Tokenize into sentences
sentences = tokenizer.to_sentences(text)

# Display tokenized words for each sentence
for sentence in sentences:
    words = tokenizer.to_words(sentence)
    print(words)
```

Python

The notebook tokenizes sample text into sentences and words. This helps demonstrate how the input is broken down before summarization.

6.2 Stemming

Stemming reduces words to a root-like form. This helps summarize related terms together, such as programming, programmed, and programmer.

```
from sumy.nlp.stemmers import Stemmer

# Create stemmer for English
stemmer = Stemmer("en")

# Test stemming on various words
test_words = ["programming", "developer", "coding", "algorithms"]

for word in test_words:
    stemmed = stemmer(word)
    print(f"{word} -> {stemmed}")
```

Python

```
programming -> program
developer -> develop
coding -> code
algorithms -> algorithm
```

7. Summarization Algorithms Used

Algorithm	Type	How it Works
Luhn Summarizer	Extractive, frequency-based	Ranks sentences based on important word frequency and sentence significance.
Edmundson Summarizer	Extractive, weighted-word method	Uses bonus words, stigma words, and null words to influence sentence selection.
LSA Summarizer	Extractive, semantic method	Uses latent semantic analysis to identify conceptually important sentences.

7.1 Luhn Summarizer Function

The Luhn Summarizer is an extractive summarization technique. It selects the most important sentences from the text based on word frequency. The main idea is that sentences containing frequently occurring important words are more likely to represent the main meaning of the document.

```
from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.luhn import LuhnSummarizer
from sumy.nlp.stemmers import Stemmer
from sumy.utils import get_stop_words
import nltk
nltk.download('punkt_tab')

def luhn_summarize(text, sentence_count=2):
    # Parse the input text
    parser = PlaintextParser.from_string(text, Tokenizer("english"))

    # Initialize summarizer with stemmer
    summarizer = LuhnSummarizer(Stemmer("english"))
    summarizer.stop_words = get_stop_words("english")

    # Generate summary
    summary = summarizer(parser.document, sentence_count)
    return summary
```

```
Machine learning algorithms form the backbone of most AI applications today.
Deep learning, a subset of machine learning, uses neural networks to solve complex problems.
[nltk_data] Downloading package punkt_tab to
[nltk_data]   C:\Users\Hp\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

7.2 Edmundson Summarizer Function

The Edmundson Summarizer is also an extractive summarization method. It improves sentence selection by using important keywords, known as bonus words, and less important words, known as stigma words. Sentences containing bonus words are given higher importance, while sentences containing stigma words may receive lower importance.

```

from sumy.summarizers.edmundson import EdmundsonSummarizer

def edmundson_summarize(text, sentence_count=2, bonus_words=None, stigma_words=None):
    parser = PlaintextParser.from_string(text, Tokenizer("english"))

    # Initialize summarizer
    summarizer = EdmundsonSummarizer(Stemmer("english"))
    summarizer.stop_words = get_stop_words("english")

    # Set null words
    summarizer.null_words = get_stop_words("english")

    # Set custom word weights
    if bonus_words:
        summarizer.bonus_words = bonus_words
    if stigma_words:
        summarizer.stigma_words = stigma_words

    summary = summarizer(parser.document, sentence_count)
    return summary

```

Python

Artificial intelligence represents a paradigm shift in how machines process information.
Machine learning algorithms form the backbone of most AI applications today.

7.3 LSA Summarizer Function

The LSA Summarizer uses Latent Semantic Analysis to identify the hidden meaning and relationships between words and sentences. It converts the text into a mathematical form and selects sentences that best represent the main concepts of the document.

```

from sumy.summarizers.lsa import LsaSummarizer

def lsa_summarize(text, sentence_count=2):
    parser = PlaintextParser.from_string(text, Tokenizer("english"))

    # Initialize LSA summarizer
    summarizer = LsaSummarizer(Stemmer("english"))
    summarizer.stop_words = get_stop_words("english")

    summary = summarizer(parser.document, sentence_count)
    return summary

sample_text = """

```

Python

Artificial intelligence represents a paradigm shift in how machines process information.
Modern AI systems can learn from data, recognize patterns, and make decisions with minimal human intervention.

8. Abstractive vs Extractive Summarization

Aspect	Hugging Face T5	Sumy Algorithms
Method	Abstractive	Extractive
Output	Can create new wording	Selects sentences from original text
Model requirement	Pre-trained neural model	Statistical or semantic sentence ranking
Speed	Usually slower due to model inference	Usually faster and lightweight
Best use case	Human-like summaries	Transparent sentence-based summaries

9. How to Run the Notebook

1. Open the notebook in Jupyter Notebook, JupyterLab, Google Colab, or VS Code.
2. Run the installation cells first to install transformers, torch, and sumy.
3. Run the Hugging Face cells in order to load the model and generate the T5 summary.
4. Run the Sumy cells in order to test tokenization, stemming, and extractive summarization algorithms.
5. Modify the `sample_text` variable to summarize a custom paragraph or article.

10. Possible Improvements

- Add input validation so empty text is handled safely.
- Wrap each summarization method into reusable functions with clear docstrings.
- Create a single comparison table showing output from T5, Luhn, Edmundson, and LSA for the same input text.
- Add evaluation metrics such as ROUGE for comparing generated summaries.
- Allow users to enter text dynamically instead of editing the code cell manually.
- Add exception handling for missing NLTK resources or failed model downloads.

11. Conclusion

The notebook successfully demonstrates both neural and traditional NLP approaches to text summarization. Hugging Face T5 is used for abstractive summarization, while Sumy provides multiple extractive summarization algorithms. Together, these examples provide a practical introduction to summarization workflows, preprocessing steps, model inference, and sentence-ranking methods.